



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN GENERAL DE PREGRADO

PROYECTO DE CARRERA: INGENIERÍA EN INFORMÁTICA

SEMESTRE: CIVA 2026

PRODUCTO I

Docente:

Marquez Félix

Integrantes:

Franner Bermudez C.I 30.183.618

Fernando Bolívar C.I 31.963.032

Miguel Mora C.I 32.466.034

Samuel Prado C.I 31.701.746

Ciudad Guayana, 2 de septiembre de 2026

Ejercicio 1:

Traducción de Necesidades a Requerimientos y Actores

Contexto: Un hospital público necesita reducir el tiempo de espera de los pacientes en urgencias. **Instrucciones:** Identifique al menos 4 actores (stakeholders) clave en este escenario. Para cada actor, redacte una "Necesidad" (el problema encontrado) y tradúzcala a un Requerimiento específico del software (funcional o no funcional).

Respuesta:

Actores:

- **Personal de enfermería de triaje:** Es el profesional de enfermería especializado que establece un primer contacto y establece un punto de entrada con el paciente para dar una primera valoración clínica.
 - **Necesidad:** Clasificar a los pacientes según el nivel de gravedad de su valoración clínica de forma rápida y efectiva.
 - **Requerimiento:** El sistema debe permitir al enfermero registrar la clasificación de la gravedad del paciente (por ejemplo en la escala de Manchester) a través de una interfaz interactiva. Además, una vez registrado, el registro es insertado en una cola de espera, y se ordena automáticamente según la clasificación clínica de cada paciente. El criterio de desempate es el orden de llegada de los pacientes en caso de tener la misma clasificación como segundo criterio de orden.
- **Paciente:** Es la persona que acude al hospital público en busca de atención médica y la cual experimenta directamente el tiempo de espera.
 - **Necesidad:** No quiere experimentar incertidumbre de su tiempo de espera o cuando será atendido, lo que genera ansiedad al no tener una noción del tiempo necesario para ser entendida, lo que incluso podría llevar a una percepción de mala calidad del servicio.
 - **Requerimiento:** El paciente una vez registrado por el personal de enfermería de triaje, debe poder visualizar en el sistema su posición en la cola de espera, la cual muestra cómo están organizados según su prioridad y secundariamente por su orden de llegada. Adicionalmente, se debe poder mostrar un mensaje del motivo de una demora anormal, en caso de (por ejemplo un retraso por la falta de personal).

- **Gestión administrativa del hospital:** Es el personal responsable de supervisar el desempeño del servicio de urgencias y a su vez gestionar los recursos (como personal, insumos, turnos) a partir de los datos que almacena el sistema.
 - **Necesidad:** Obtener información clara y actualizada sobre los tiempos de espera y pacientes atendidos por turno, para así identificar cuellos de botella o deficiencias en la metodología actual sin tener que duplicar esta información o solo ser accesible en registros manuales.
 - **Requerimiento:** El sistema debe generar automáticamente reportes con indicadores clave (o KPIs) como tiempo de espera, volumen de pacientes por turno, etc. Este reporte debe ser exportable en formatos comunes como PDF o CSV, para su entendimiento sin requerir conocimientos técnicos de consultas de bases de datos.
- **Médico de urgencias:** Es quien atiende al paciente ya clasificado y decide cual es el diagnóstico o tratamiento correspondiente.
 - **Necesidad:** No perder tiempo en las tareas administrativas repetitivas durante la consulta (como los datos ya capturados en el triaje), ya que cada minuto perdido por paciente acrecienta el tiempo de espera de los demás.
 - **Requerimiento:** El sistema debe mostrar al médico en el inicio de la consulta los datos ya capturados en el triaje o incluso en recepción sin necesidad de volver a solicitarlos o anotarlos manualmente.

Estándar y visión aplicada: Para la construcción de los requerimientos se aplicó la norma ISO/IEC/IEEE 29148, asegurando que cada requerimiento fuera unívoco, verificable y completo (incluyendo casos borde como el desempate en la clasificación de triaje). Asimismo, se mantuvo la separación entre requerimiento, es decir, el comportamiento esperado del sistema, y la solución técnica de implementación, evitando incorporar estructuras de datos o decisiones de arquitectura en la fase de elicitación de requisitos.

Reflexión: ¿Cómo podría la IA o el prototipado ayudar a validar estos requisitos antes de programar?

El propósito del prototipado es esencialmente validar los requerimientos contra los actores relacionados al sistema, por lo que se podría elaborar una maqueta interactiva probada con personal de enfermería real para revelar si el registro de clasificación realmente es eficiente, o si el diseño propuesto es más lento de lo esperado en la práctica. Paralelamente, un prototipo de la pantalla del paciente permite comprobar si realmente esta tiene el efecto esperado de reducir su percepción de espera o mejorar sus decisiones antes de entrar a consulta.

Por otro lado, la IA generativa puede reducir el tiempo necesario para generar un mockup o un prototipado rápido para mostrar a los actores. Además de que permite detectar ambigüedades o casos borde según criterios de la norma ISO/IEC/IEEE 29148, o generar posibles variantes del sistema para tener distintas opciones que sean validadas por el programador que tiene el contexto real.

Ejercicio 2:

Análisis de las 4 Dimensiones y Errores Clásicos

Contexto: Una startup decide desarrollar una app de delivery. El gerente técnico impone el uso de una arquitectura de microservicios en Kubernetes (Tecnología), pero el equipo solo sabe programar en PHP monolítico (Personas). No definieron un proceso de gestión de cambios (Proceso) y el cliente pide agregar una función de "pagos con criptomonedas" a mitad del desarrollo sin ajustar el presupuesto (Producto/Alcance). **Instrucciones:** Analice el caso identificando los desbalances en las 4 dimensiones de Pressman. Identifique al menos 3 "Errores clásicos" que se están cometiendo en este escenario. Proponga una solución de mitigación inmediata desde el rol de un Gerente de Proyectos moderno (Líder Servicial).

Respuesta: En esta startup, la falta de alineación entre las cuatro dimensiones del proyecto (personas, proceso, producto y tecnología) pone en riesgo directo el éxito del desarrollo. Para empezar existe una desconexión total entre las personas y la tecnología: forzar a un equipo acostumbrado a desarrollar monolitos en PHP a adoptar de golpe microservicios con Kubernetes genera una brecha técnica inmanejable a corto plazo, ignorando que el factor humano es el pilar más crítico del sistema. A esto se suma la ausencia de un proceso de gestión de cambios, lo que deja al producto desprotegido ante exigencias imprevistas del cliente. Sin un filtro

metodológico claro, se cae inevitablemente en varios de los "errores clásicos" de la ingeniería de software. El primero es el descontrol del alcance o Scope Creep, al aceptar la función de pagos con criptomonedas a mitad de camino sin ajustar el presupuesto ni los plazos. El segundo es un marcado optimismo injustificado, al asumir que el equipo asimilará una infraestructura tan compleja sobre la marcha y sin retrasos. Y el tercero es la falta de una gestión real de requisitos, pues se introducen nuevas características de manera improvisada y sin analizar el valor real que aportan al negocio en esta fase.

Desde el rol de la gerencia de proyectos bajo un enfoque de liderazgo servicial, la prioridad no es imponer control jerárquico, sino remover obstáculos, proteger al equipo y asegurar que se entregue valor de forma rápida y sostenible. Como acción inmediata, se propone coordinar con el liderazgo técnico para simplificar la arquitectura, optando por un monolito modular en PHP que el equipo ya domine por completo. Esto permitirá acelerar el lanzamiento de la aplicación y reducir el tiempo de salida al mercado, dejando la migración a microservicios como un plan de capacitación a mediano plazo y sin presión de entrega. En paralelo, se debe abrir un canal de comunicación transparente con el cliente para explicar el impacto técnico y proponer el manejo de la funcionalidad de criptomonedas como una adición para una segunda fase. De este modo, se prioriza un Producto Mínimo Viable (MVP) funcional que valide el negocio sin asfixiar a los desarrolladores. Finalmente, se establece un proceso ágil ligero donde se defina formalmente que cualquier cambio futuro deba ser evaluado, estimado y priorizado en conjunto antes de ser desarrollado, devolviendo el orden y la seguridad al proyecto.

Ejercicio 3:

Explique la diferencia entre la gestión de proyectos bajo el enfoque del PMBOK 6 (basado en procesos) y el PMBOK 7 (basado en principios). ¿Cómo cambia esto la forma en que gestionamos los requisitos del software? Explique mediante 2 ejemplos.

Respuesta: La diferencia fundamental entre ambas ediciones radica en el cambio de una mentalidad prescriptiva a una orientada al valor.

El PMBOK 6 estructura la gestión de proyectos a través de 49 procesos organizados rígidamente en 5 grupos de procesos y 10 áreas de conocimiento. En este esquema, el éxito se mide por el cumplimiento estricto del plan, las herramientas, las entradas y salidas, y el control de la triple restricción tradicional (alcance, tiempo y costo).

Por el contrario, el PMBOK 7 abandona el recetario de procesos para basarse en 12 principios rectores y 8 dominios de desempeño. Este nuevo enfoque es agnóstico respecto a la metodología (sea predictiva, ágil o híbrida) y se centra en los resultados (*outcomes*) y la entrega continua de valor comercial, promoviendo la adaptabilidad del equipo frente a la incertidumbre y el contexto del negocio.

Este cambio transforma radicalmente la forma en que se gestionan los requisitos de software. Bajo el PMBOK 6, los requisitos se tratan como una línea base fija y formal desde el inicio del proyecto. Se documentan de manera exhaustiva en especificaciones detalladas y matrices de trazabilidad, donde cualquier modificación requiere pasar por un riguroso Comité de Control de Cambios (CCB) para evitar desviaciones en el alcance. En cambio, bajo el PMBOK 7, los requisitos se gestionan como necesidades vivas y emergentes. Se prioriza el aprendizaje continuo, los ciclos rápidos de retroalimentación y la evolución constante del *backlog* en función del valor que percibe el usuario final, permitiendo que el alcance se ajuste dinámicamente sin trámites burocráticos innecesarios.

Ejemplo 1: Autenticación en una app móvil

- **PMBOK 6:** Se define por contrato un registro con correo y contraseña. Si los usuarios piden acceso biométrico (huella o rostro), se debe pausar el avance y tramitar una solicitud formal de control de cambios para no alterar la línea base del alcance.
- **PMBOK 7:** El requisito busca un *acceso seguro y rápido*. Al ver que el registro tradicional causa abandono, el equipo ajusta el *backlog* e implementa la biometría en el siguiente sprint para maximizar el valor del producto.

Ejemplo 2: Módulo de reportes en un sistema web

- **PMBOK 6:** La especificación exige programar 20 formatos de reportes en PDF. Aunque el cliente luego confirme que solo usa 2 métricas, el equipo debe construir los 20 para cumplir con el alcance pactado.
- **PMBOK 7:** Se lanza primero un panel con los datos esenciales. Al comprobar qué métricas realmente necesita el usuario, se descartan los reportes sobrantes y se redirige el esfuerzo a mejorar el rendimiento del sistema.

Ejercicio 4:

En la ingeniería de software moderna, el mayor riesgo no es que el código no compile, sino que el software se construya perfectamente pero nadie lo use. Discuta esta afirmación relacionándola con la importancia de la ingeniería de requisitos, la identificación de actores y la crisis del software.

Respuesta: La afirmación refleja cómo ha evolucionado la "crisis del software" con el paso del tiempo. En los inicios de la informática, el mayor reto era puramente técnico: lograr que los programas funcionaran sin errores y aprovecharan los pocos recursos de las computadoras. Hoy en día, gracias a las herramientas y lenguajes modernos, hacer que el código compile y se ejecute correctamente es lo habitual. El verdadero problema actual es el riesgo de construir un sistema que nadie necesite, desperdiciando tiempo y dinero en un producto que no aporta valor real al negocio, por muy bien programado que esté.

Este error suele ocurrir cuando falla la identificación de actores (stakeholders) al inicio del proyecto. Si los desarrolladores solo escuchan a la gerencia o al cliente que paga, y se olvidan de los usuarios finales que operarán el sistema todos los días, el resultado será un fracaso. Un software diseñado sin tomar en cuenta el entorno de trabajo real y las necesidades de las personas que lo van a usar resultará incómodo o inútil. Esto provocará que los usuarios rechacen el sistema y prefieran seguir usando sus métodos anteriores, como hojas de cálculo o procesos manuales.

Para evitar que el software quede en el abandono, la ingeniería de requisitos es fundamental, especialmente para entender la diferencia entre verificar y validar. La

verificación solo confirma que construimos el sistema correctamente (que no tenga errores de código y cumpla los planos técnicos). Sin embargo, la validación se asegura de que estemos construyendo el producto correcto (que realmente resuelva el problema del usuario). Al hacer un buen levantamiento de requisitos, el equipo de desarrollo se asegura de entender qué necesitan realmente los actores antes de empezar a programar, garantizando que el sistema final sea verdaderamente útil.